

Bài Tập Cuối Chương

Xây Dựng Hệ Thống Authentication Cho Ứng Dụng Quản Lý Ghi Chú (Note App)

Mục tiêu

- Áp dụng kiến thức về **Authentication** và **Authorization** sử dụng **JWT** trong React.
- Tích hợp **Redux Toolkit** để quản lý trạng thái xác thực và dữ liệu ghi chú.
- Tạo các **Protected Routes** để bảo vệ các trang yêu cầu đăng nhập.
- Xử lý các trường hợp như **Access Token hết hạn**.

Yêu cầu

Xây dựng một ứng dụng **Note App** với các tính năng sau (chỉ tập trung vào frontend, backend được cung cấp sẵn):

1. **Đăng ký (Signup):** Người dùng có thể tạo tài khoản mới với **email** và **password**.
2. **Đăng nhập (Login):** Người dùng đăng nhập bằng **email** và **password** để nhận **JWT Access Token**.
3. **Đăng xuất (Logout):** Xóa Access Token và chuyển hướng người dùng về trang đăng nhập.
4. **Quản lý ghi chú (Notes):**
 - Người dùng đã đăng nhập có thể tạo, xem, chỉnh sửa, xóa ghi chú.
 - Ghi chú bao gồm: **id**, **title**, **content**, **createdAt**.
 - Chỉ người dùng đã đăng nhập mới được truy cập trang quản lý ghi chú.
5. **Bảo vệ route:** Trang quản lý ghi chú (**/notes**) chỉ cho phép truy cập nếu người dùng đã đăng nhập.
6. **Xử lý token hết hạn:** Nếu Access Token hết hạn, tự động đăng xuất và chuyển hướng về trang đăng nhập.
7. **Giao diện đơn giản:** Không cần quá phức tạp, tập trung vào logic xác thực và quản lý ghi chú.

Cấu trúc thư mục

Ứng dụng frontend phải tuân theo cấu trúc thư mục sau:

```
note-app/
├── src/
│   ├── api/
│   │   └── axiosClient.js
│   ├── components/
│   │   ├── NoteInput/
│   │   │   └── NoteInput.jsx
│   │   ├── NoteItem/
│   │   │   └── NoteItem.jsx
│   │   ├── NoteList/
│   │   │   └── NoteList.jsx
│   │   ├── LogoutButton/
│   │   │   └── LogoutButton.jsx
│   │   └── ProtectedRoute/
│   │       └── ProtectedRoute.jsx
│   ├── redux/
│   │   ├── slices/
│   │   │   ├── authSlice.js
│   │   │   └── noteSlice.js
│   │   └── store.js
│   ├── screens/
│   │   └── Home/
│   │       └── Home.jsx
```

```
| | | Login/
| | |   Login.jsx
| | | Register/
| | |   Register.jsx
| | | Notes/
| | |   Notes.jsx
| | App.jsx
| | main.jsx
| .env
| .gitignore
```

Backend (Cung cấp sẵn)

Hướng dẫn thiết lập backend

Học viên chỉ cần chạy backend đã được cung cấp để sử dụng các API. Thực hiện các bước sau:

1. **Tạo thư mục backend:**

```
mkdir note-app-backend
cd note-app-backend
```

2. **Khởi tạo dự án Node.js:**

```
npm init -y
```

3. **Cài đặt thư viện cần thiết:**

```
npm install express cors jsonwebtoken
```

4. **Tạo file `server.js` :**

- Tạo file `server.js` trong thư mục `note-app-backend` và sao chép đoạn code sau:

```
const express = require('express');
const cors = require('cors');
const jwt = require('jsonwebtoken');

const app = express();
const JWT_SECRET = 'NOTE_APP_SECRET';
app.use(cors({ origin: 'http://localhost:3000' }));
app.use(express.json());

let users = [];
let notes = [];

const authenticateJWT = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token provided' });
  jwt.verify(token, JWT_SECRET, (err, user) => {
    if (err) return res.status(403).json({ message: 'Invalid token' });
    req.user = user;
    next();
  });
};

app.post('/api/signup', (req, res) => {
  const { email, password } = req.body;
  if (users.find((u) => u.email === email)) {
    return res.status(409).json({ message: 'Email already exists' });
  }
});
```

```

    }
    users.push({ email, password });
    res.status(201).json({ message: 'User registered' });
  });

app.post('/api/login', (req, res) => {
  const { email, password } = req.body;
  const user = users.find((u) => u.email === email && u.password === password);
  if (!user) return res.status(401).json({ message: 'Invalid credentials' });
  const token = jwt.sign({ email: user.email }, JWT_SECRET, { expiresIn: '30m' });
  res.json({ accessToken: token });
});

app.get('/api/notes', authenticateJWT, (req, res) => {
  const userNotes = notes.filter((note) => note.userEmail === req.user.email);
  res.json(userNotes);
});

app.post('/api/notes', authenticateJWT, (req, res) => {
  const { title, content } = req.body;
  const newNote = {
    id: Date.now(),
    title,
    content,
    userEmail: req.user.email,
    createdAt: new Date().toISOString(),
  };
  notes.push(newNote);
  res.status(201).json(newNote);
});

app.patch('/api/notes/:id', authenticateJWT, (req, res) => {
  const { id } = req.params;
  const { title, content } = req.body;
  const noteIndex = notes.findIndex(
    (note) => note.id === parseInt(id) && note.userEmail === req.user.email
  );
  if (noteIndex === -1) return res.status(404).json({ message: 'Note not found' });
  notes[noteIndex] = { ...notes[noteIndex], title, content };
  res.json(notes[noteIndex]);
});

app.delete('/api/notes/:id', authenticateJWT, (req, res) => {
  const { id } = req.params;
  const noteIndex = notes.findIndex(
    (note) => note.id === parseInt(id) && note.userEmail === req.user.email
  );
  if (noteIndex === -1) return res.status(404).json({ message: 'Note not found' });
  notes.splice(noteIndex, 1);
  res.status(204).send();
});

app.listen(5000, () => console.log('Server running on <http://localhost:5000>'));

```

5. Chạy backend:

```
node server.js
```

- Backend sẽ chạy tại `http://localhost:5000`.

- Đảm bảo backend đang chạy trước khi khởi động frontend.

Các API có sẵn

- **POST /api/signup:**
 - Body: `{ "email": "string", "password": "string" }`
 - Trả về: `201` nếu đăng ký thành công, `409` nếu email đã tồn tại.
- **POST /api/login:**
 - Body: `{ "email": "string", "password": "string" }`
 - Trả về: `{ "accessToken": "string" }` nếu thành công, `401` nếu thông tin sai.
- **GET /api/notes** (yêu cầu JWT):
 - Header: `Authorization: Bearer <token>`
 - Trả về: Danh sách ghi chú của người dùng.
- **POST /api/notes** (yêu cầu JWT):
 - Header: `Authorization: Bearer <token>`
 - Body: `{ "title": "string", "content": "string" }`
 - Trả về: Ghi chú vừa tạo.
- **PATCH /api/notes/:id** (yêu cầu JWT):
 - Header: `Authorization: Bearer <token>`
 - Body: `{ "title": "string", "content": "string" }`
 - Trả về: Ghi chú đã cập nhật.
- **DELETE /api/notes/:id** (yêu cầu JWT):
 - Header: `Authorization: Bearer <token>`
 - Trả về: `204` nếu xóa thành công.

Hướng dẫn Frontend

1. Thiết lập môi trường

- Tạo dự án React sử dụng Vite:

```
npm create vite@latest note-app -- --template react
cd note-app
npm install
```

- Cài đặt thư viện cần thiết:

```
npm install react-router-dom @reduxjs/toolkit react-redux axios
```

2. Yêu cầu Frontend

- **api/axiosClient.js:** Cấu hình Axios với:
 - Request interceptor để đính kèm JWT vào header `Authorization`.
 - Response interceptor để xử lý lỗi 401 (token hết hạn) bằng cách đăng xuất.
- **components/:**
 - **NoteInput/NoteInput.jsx:** Form để tạo hoặc chỉnh sửa ghi chú (title, content).
 - **NoteItem/NoteItem.jsx:** Hiển thị một ghi chú với các nút chỉnh sửa/xóa.
 - **NoteList/NoteList.jsx:** Hiển thị danh sách ghi chú, sử dụng NoteItem.
 - **LogoutButton/LogoutButton.jsx:** Nút để đăng xuất.
 - **ProtectedRoute/ProtectedRoute.jsx:** Bảo vệ các route yêu cầu đăng nhập.

- **redux/:**
 - **slices/authSlice.js:** Quản lý đăng ký, đăng nhập, đăng xuất.
 - **slices/noteSlice.js:** Quản lý các thao tác CRUD cho ghi chú (lấy danh sách, tạo, cập nhật, xóa).
 - **store.js:** Kết hợp authSlice và noteSlice.
- **screens/:**
 - **Home/Home.jsx:** Trang chào mừng hoặc chuyển hướng đến /notes.
 - **Login/Login.jsx:** Form đăng nhập với email, password.
 - **Register/Register.jsx:** Form đăng ký với email, password.
 - **Notes/Notes.jsx:** Trang chính hiển thị NoteInput và NoteList.
- **App.jsx:** Thiết lập các route:
 - `/login` : Trang đăng nhập.
 - `/register` : Trang đăng ký.
 - `/notes` : Trang quản lý ghi chú (bảo vệ bằng ProtectedRoute).
 - `/` : Trang Home hoặc chuyển hướng đến /notes.
- **.env:** Cấu hình biến môi trường:

```
VITE_API_BASE_URL=http://localhost:5000/api
```

3. Bài tập chi tiết

1. Cấu hình và kết nối API:

- Thiết lập `axiosClient.js` để gọi các API backend đã cung cấp.
- Đảm bảo gửi JWT trong header cho các API yêu cầu xác thực.
- Xử lý lỗi 401 để đăng xuất khi token hết hạn.

2. Quản lý trạng thái:

- Tạo `authSlice.js` với các async thunk cho đăng ký, đăng nhập, đăng xuất.
- Tạo `noteSlice.js` với các async thunk cho lấy danh sách, tạo, cập nhật, xóa ghi chú.

3. Giao diện và điều hướng:

- Thiết kế giao diện đơn giản cho `Login` , `Register` , và `Notes` .
- Tích hợp `ProtectedRoute` để bảo vệ trang `/notes` .
- Đảm bảo chuyển hướng đúng khi đăng nhập, đăng xuất, hoặc token hết hạn.

4. Kiểm tra:

- Đăng ký tài khoản mới và đăng nhập thành công.
- Tạo, chỉnh sửa, xóa ghi chú, xác nhận chỉ thấy ghi chú của mình.
- Đăng xuất và xác nhận không thể truy cập trang `/notes` .
- Đợi token hết hạn (30 phút) và nếu có ứng dụng có gọi protected API thì tự động chuyển hướng về trang đăng nhập.